

Advanced Data Analytics — Lecture 10

Introduction to Gaussian Process Regression

Simon Scheidegger

Department of Economics, University of Lausanne, Switzerland

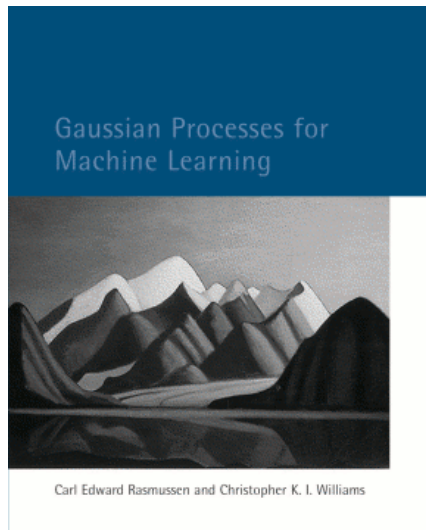
November 17th, 2025 | 10:15 - 12:00: Internef 126 | 16:30 - 18:00: Anthropole 3185

Gaussian Process Regression

- ▶ Today in the first part:
 - ▶ Basics of Gaussian Process Regression
 - ▶ Noise-free kernels

Gaussian Process Regression

<http://www.gaussianprocess.org/gpml/>

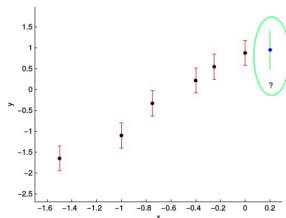


Recall: Aim of Regression

- ▶ Given some (potential) noisy **observations** of a dependent variable at certain values of the **independent variable** x , what is our **best estimate of the dependent variable** y **at a new value**, x_* ?
- ▶ Let f denote an (unknown) function which maps inputs x to outputs

$$f: X \rightarrow Y$$

- ▶ Modeling a function f means mathematically representing the relation between inputs and outputs.
- ▶ Often times, the shape of the underlying function might be unknown, the function can be hard to evaluate, or other requirements might complicate the process of information acquisition.



Choosing a model

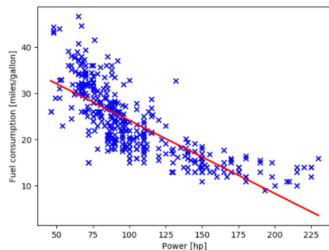
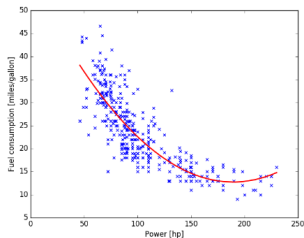
- ▶ If we expect the underlying function $f(x)$ to be linear, and can make some assumptions about the input data, we might use a least-squares method to fit a straight line (linear regression).
- ▶ Moreover, if we suspect $f(x)$ may also be quadratic, cubic, or even non-polynomial, we can use the principles of model selection to choose among the various possibilities.

Model Selection

Example data set by <https://archive.ics.uci.edu/ml/datasets/Auto+MPG>

One common approach to reliably assess the quality of a machine learning model and avoid over-fitting is to **randomly split the available data** into

- ▶ **training data** (70% of the data) is used for determining optimal coefficients.
- ▶ **validation data** (20% of the data) is used for **model selection** (e.g., fixing degree of polynomial, selecting a subset of features, etc.)
- ▶ **test data** ($\sim 10\%$ of the data) is used to measure the quality that is reported.



For completeness: Polynomial Regression in Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model, preprocessing

# read cars dataset, a clean data set
cars = pd.read_csv('auto-mpg.data.txt', header=None, sep='\s+')

# extract mpg values
y = cars.iloc[:,0].values

# extract horsepower values
X = cars.iloc[:,[3]].values
X = X.reshape(X.size, 1)

# precompute polynomial features
poly = preprocessing.PolynomialFeatures(2)
Xp = poly.fit_transform(X)

# fit linear regression model
reg = linear_model.LinearRegression()
reg.fit(Xp,y)
```

```
# coefficients
reg.intercept_ # 56.900099702112925
reg.coef_ # [-0.46618963, 0.00123054]

# compute correlation coefficient
np.corrcoef(reg.predict(Xp),y) # 0.82919179 (from 0.77842678)

# compute mean squared error (MSE)
sum((reg.predict(Xp) - y)**2) / len(y) # 18.984768907617223
# ( from 23.943662938603104)

### plot

hp = cars.iloc[:,3].values
mpg = cars.iloc[:,0].values

hps = np.array(sorted(hp))
hps = hps.reshape(hps.size, 1)
hpsp = poly.fit_transform(hps)

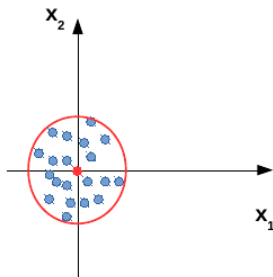
plt.scatter(hp, mpg, color='blue', marker='x')
plt.plot(hps, reg.predict(hpsp), color='red', lw=2)
plt.xlabel('Power [hp]')
plt.ylabel('Fuel consumption [miles/gallon]')
plt.show()
```

Why Gaussian Process Regression?

- ▶ There are **many projections possible**.
- ▶ We have to choose one either a priori or by model comparison with a set of possible projections.
- ▶ Especially if the problem is to **explore and exploit a completely unknown function**, this approach will not be beneficial as there is little guidance to which projections we should try.

Gaussian process regression offers a principled solution to this problem in which projections are chosen implicitly, effectively letting “the data decide” on the complexity of the function.

Recall: Multivariate Gaussians

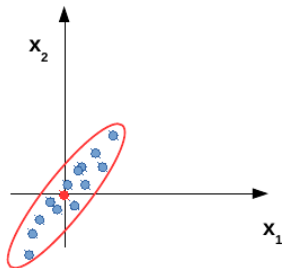


Say you measure two variables, e.g.,

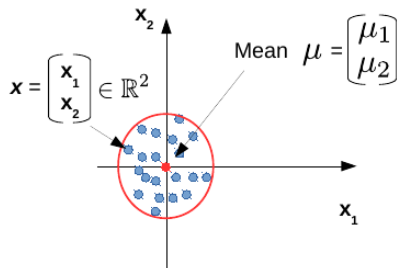
- x_1 : height
- x_2 : weight

→ plot

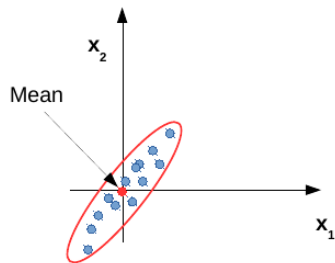
→ we want to fit a Gaussian to these points.



Multivariate Gaussians (II)



Fit a Gaussian that with a covariance that is **circular**.



Fit a Gaussian that with a covariance that is an **ellipse**.

Multivariate Gaussians (III)

- ▶ Assume the points are Gaussian distributed (this is our "model").
- ▶ How do points relate to each other? ("how does increasing x_1 increase x_2 ?)
- ▶ The variable to describe this is called "Covariance" (cov)
If the entries in the column vector $\mathbf{x} = (X_1, X_2, \dots, X_n)^T$ are random variables, each with finite variance and expected value, then the covariance matrix $K_{\alpha\alpha}$ is the matrix whose (i, j) entry is the covariance.

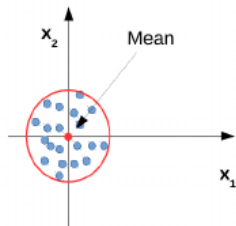
$$K_{X_i X_j} = \text{cov}[X_i, X_j] = E[(X_i - E[X_i])(X_j - E[X_j])]$$

Multivariate Gaussians (IV)

- ▶ Assume for a moment that the Covariance $E[x_1x_2]$ is a “dot” product.
- ▶ Assume two points $\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$ – Covariance is a measure similarity.

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \mathcal{N} \left[\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right]$$
$$E[x_1x_2] = 0$$

Knowing about x_1 does not provide any information about x_2 as they are uncorrelated.



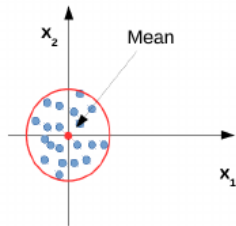
Multivariate Gaussians (V)

- ▶ Assume for a moment that $E[x_1 x_2] = 0$ — Covariance $E[x_1 x_2]$ is a “dot” product.
- ▶ Assume two points $\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$ — Covariance is a measure similarity.

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim N \left[\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix} \right]$$

$$E[x_1 x_2] \neq 0$$

- ▶ **Knowing about x_1 DOES provide information about x_2 .**
- ▶ if x_1 is positive, x_2 is with great probability.
- ▶ knowing something about x_1 allows us to know something about x_2 .

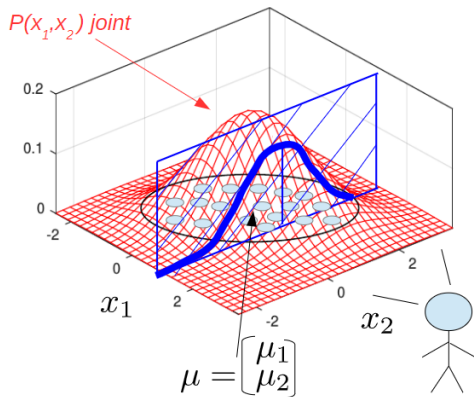


Joint Gaussian distributions

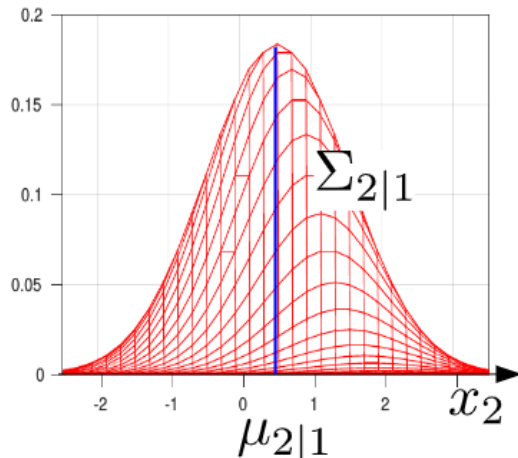
see, e.g., Rasmussen et al. (2005), Murphy (2012)

Conditional distribution

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim N \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right)$$



$$P(x_2 | X_1 = x_1)$$



From Joint to Conditional distributions

see, e.g., Murphy (2012), chapter 4

Theorem 4.3.1 (Marginals and conditionals of an MVN). Suppose $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2)$ is jointly Gaussian with parameters

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix}, \quad \boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1} = \begin{pmatrix} \boldsymbol{\Lambda}_{11} & \boldsymbol{\Lambda}_{12} \\ \boldsymbol{\Lambda}_{21} & \boldsymbol{\Lambda}_{22} \end{pmatrix} \quad (4.67)$$

Then the marginals are given by

$$\begin{aligned} p(\mathbf{x}_1) &= \mathcal{N}(\mathbf{x}_1 | \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_{11}) \\ p(\mathbf{x}_2) &= \mathcal{N}(\mathbf{x}_2 | \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_{22}) \end{aligned} \quad (4.68)$$

and the posterior conditional is given by

$$\begin{aligned} p(\mathbf{x}_1 | \mathbf{x}_2) &= \mathcal{N}(\mathbf{x}_1 | \boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2}) \\ \boldsymbol{\mu}_{1|2} &= \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \\ &= \boldsymbol{\mu}_1 - \boldsymbol{\Lambda}_{11}^{-1} \boldsymbol{\Lambda}_{12} (\mathbf{x}_2 - \boldsymbol{\mu}_2) \\ &= \boldsymbol{\Sigma}_{1|2} (\boldsymbol{\Lambda}_{11} \boldsymbol{\mu}_1 - \boldsymbol{\Lambda}_{12} (\mathbf{x}_2 - \boldsymbol{\mu}_2)) \\ \boldsymbol{\Sigma}_{1|2} &= \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21} = \boldsymbol{\Lambda}_{11}^{-1} \end{aligned} \quad (4.69)$$

Two "blocks" of vectors

This Theorem allows you to go from joint to conditional distributions.

Producing Data from Gaussians

$$x_i \sim \mathcal{N}(0, 1)$$

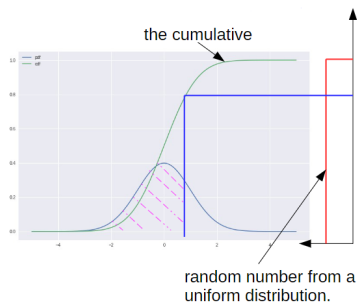
$$x_i \sim \mathcal{N}(\mu, \sigma^2) \sim \mu + \sigma \mathcal{N}(0, 1)$$

As we have the capability of drawing 1-dim random numbers from a Gaussian, we can also do this in a multivariate case.

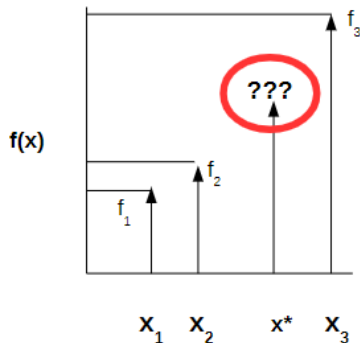
- We need a way to take “square roots” from matrices.
- Cholesky decomposition: $\Sigma = LL^T$

$$\begin{array}{c} \longrightarrow \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right) \longrightarrow x \sim \mu + L\mathcal{N}(0, \mathbf{I}) \\ \uparrow \qquad \qquad \qquad \uparrow \qquad \qquad \qquad \uparrow \\ \mathbf{x} \qquad \qquad \qquad \mu \qquad \qquad \qquad \Sigma \end{array}$$

Recall Eq. (4.68) from the previous slide.



Observations $\rightarrow$ Interpolation



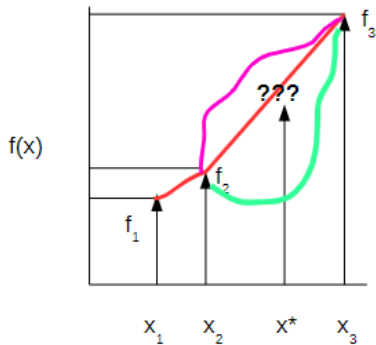
We have 3 observations at x_i for $f(x_i)$

► $\rightarrow$ Given the data pairs

$$D = \{(x_1, f_1), (x_2, f_2), (x_3, f_3)\}$$

► $\rightarrow$ we want to find/learn the function that describes the data, i.e., for a "new" x^* , we want to know what $f(x^*)$ would be!

Observations $\rightarrow$ Interpolation (II)



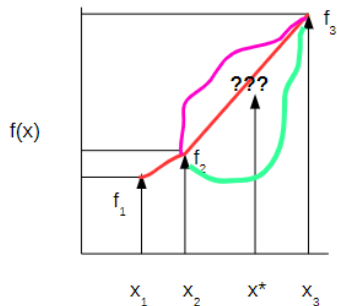
We assume that f 's (the height) are **Gaussian distributed**, with **zero - mean** and some **covariance matrix** K .

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N} \left[\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} K_{11}, K_{12}, K_{13} \\ K_{21}, K_{22}, K_{23} \\ K_{31}, K_{32}, K_{33} \end{bmatrix} \right]$$

Note: f_1 and f_2 should probably be more correlated, as they are nearby (compared to f_1 and f_3).

- The prior mean function μ reflects the expected function value at input x : $\mu(x) = \mathbb{E}(f(x))$
- It is often set to 0.

Observations $\rightarrow$ Interpolation (II)



Covariance matrix constructed by some “measure of similarity”, i.e., a kernel function (parametric ansatz), such as “squared exponential”. Parameters can be obtained e.g. via MLE (later).

We assume that f 's (the height) are Gaussian distributed, with **zero - mean** and some **covariance matrix** K .

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim N \left[\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix} \right]$$

Note: f_1 and f_2 should probably be more correlated, as they are nearby (compared to f_1 and f_3), e.g.,

$$\sim N \left(\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.7 & 0.2 \\ 0.7 & 1 & 0.6 \\ 0.2 & 0.6 & 1 \end{bmatrix} \right)$$

$$\kappa(x, x') = \sigma_f^2 \exp \left(-\frac{1}{2\ell^2} (x - x')^2 \right)$$

σ_f^2 controls vertical variation;

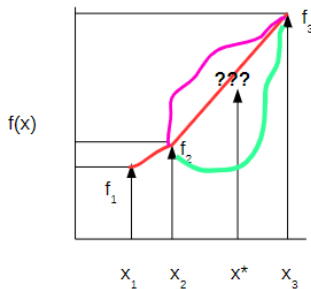
ℓ controls horizontal length scale.

Observations $\rightarrow$ Interpolation (III)

Given data $D =$

$\{(x_1, f_1), (x_2, f_2), (x_3, f_3)\} \rightarrow f(x^*) = f_* ?$

- Assume $f \sim N(0, K(\cdot, \cdot))$,
- Assume $f(x^*) \sim N(0, K(x^*, x^*))$



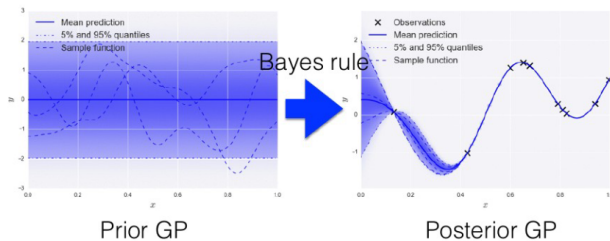
$$K(x_1, x_*) = K_{1*}$$

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_* \end{pmatrix} \sim N \left(\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \text{3d-Covariance } K \\ \text{from training data} \\ \hline \overbrace{K_{11}, K_{12}, K_{13}}^{K_{1*}}, K_{1*} \\ K_{21}, K_{22}, K_{23}, K_{2*} \\ \overbrace{K_{31}, K_{32}, K_{33}}^{K_{3*}}, K_{3*} \\ K_{*1}, K_{*2}, K_{*3}, K_{**} \end{bmatrix} \right).$$

- **Joint distribution over f and f_* .**
- **We need the conditional of f_* given f .**
- In this example, we “cut” in 3 dimensions.
- What is left is a 1-dimensional Gaussian, i.e., the Gaussian for f_*

Interpolation $\rightarrow$ Noiseless GPR

(see, e.g., Rasmussen & Williams (2006), with references therein)



Training set: $D = \{(x_i, y_i) \mid i = 1, \dots, n\}$

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{pmatrix} \right)$$

$$p(\mathbf{f}_* \mid \mathbf{X}_*, \mathbf{X}, \mathbf{f}) = \mathcal{N}(\mathbf{f}_* \mid \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*)$$

$$\boldsymbol{\mu}_* = \boldsymbol{\mu}(\mathbf{X}_*) + \mathbf{K}_*^T \mathbf{K}^{-1}(\mathbf{f} - \boldsymbol{\mu}(\mathbf{X}))$$

$$\boldsymbol{\Sigma}_* = \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_*$$

Test point = interpolation at $\mathbf{X}^*$

predictive mean $\mu_* = \mathbb{E}(f_*)$

Confidence Intervals!

Algorithmic Complexity!

- ▶ The central computational operation in using Gaussian processes will involve the **inversion of a matrix of size $N \times N$** , for which standard methods require **$O(N^3)$** computations.
- ▶ The matrix inversion must be performed **once for the given training set**.
 - ▶ → For large training data sets, however, **the direct application of Gaussian process methods can become infeasible**.
 - ▶ → Sparse Methods (cf. Rasmussen et. al (2006) and references therein).

A note on the predictive mean

Note that the predictive mean can (in general) also be written as

$$\mu = m(x) + \sum_{i=1}^N a_i k(x_i, x)$$

where $\mathbf{a} = (a_1, \dots, a_N) = (\mathbf{K} + s_n^2 \mathbf{I}_N)^{-1} (\mathbf{t} - \mathbf{m})$ and $\mathbf{t}$ being the N observations.

→ We can think of the **GP posterior mean** as an **approximation of $f(\cdot)$ using N symmetric basis functions** centered at each observed input.

→ **by choosing a covariance function that vanishes when x and x' are separated by a lot, for example the squared exponential covariance function, we see that an observed input-output will only affect the approximation locally.**

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta}) = s^2 \exp \left\{ -\frac{1}{2} \sum_{i=1}^D \frac{(x_i - x'_i)^2}{\ell_i^2} \right\}$$

GP — a distribution over functions

$$f(x) \sim GP(\mu(x), k(x, x'))$$

$$\mu(x) = \mathbb{E}[f(x)]$$

$$k(x, x') = \mathbb{E} \left[(f(x) - \mu(x)) \left((f(x') - \mu(x')) \right)^T \right]$$

$$k(x, x') = \exp \left(-\frac{1}{2} (x - x')^2 \right)$$

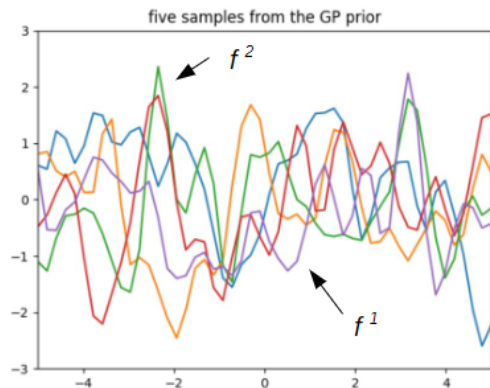
Procedure:

Create vector $X_{1:N}$

$$\mu = 0, K_{N \times N}$$

$$K = LL^T$$

$$f \sim N(0, K) \sim LN(0, I)$$



GPR Example code & numerical stability

(demo/1d_gp_example.ipynb and Murphy, Chapter 15)

```
import numpy as np
import matplotlib.pyplot as plt
# We assume a zero mean GP Prior
# This is the true unknown, one-dimensional function we are trying to approximate
f = lambda x: np.sin(0.9*x).flatten()
# This is the kernel function
def kernel_function(a, b):
    """ GP squared exponential kernel function """
    kernelParameter = 1.0
    sqdist = np.sum(a**2,1).reshape(-1,1) + np.sum(b**2,1) - 2*np.dot(a, b.T)
    return np.exp(-.5 * (1/kernelParameter) * sqdist)
# Here we run the test
N = 10          # number of training points.
n = 50          # number of test points.
s = 0.5         # noise variance.
# Sample some input points and noisy versions of the function evaluated at these points.
X = np.random.uniform(-5, 5, size=(N,1))
y = f(X) + s*np.random.randn(N) # add some noise
K = kernel_function(X, X)
L = np.linalg.cholesky(K + s*np.eye(N))
# points we're going to make predictions at.
TestPoint = np.linspace(-5, 5, n).reshape(-1,1)
# compute the mean at our test points.
Lk = np.linalg.solve(L, kernel_function(X, TestPoint))
mu = np.dot(Lk.T, np.linalg.solve(L, y))
# compute the variance at our test points.
K_ = kernel_function(TestPoint, TestPoint)
s2 = np.diag(K_) - np.sum(Lk**2, axis=0)
s = np.sqrt(s2)
```

$$\begin{aligned}\mathbb{E}[f(x_*)] &= \mu = k_*^T K_y^{-1} y \\ K_y &= LL^T \\ \alpha &= K_y^{-1} y = L^{-T} L^{-1} y\end{aligned}$$

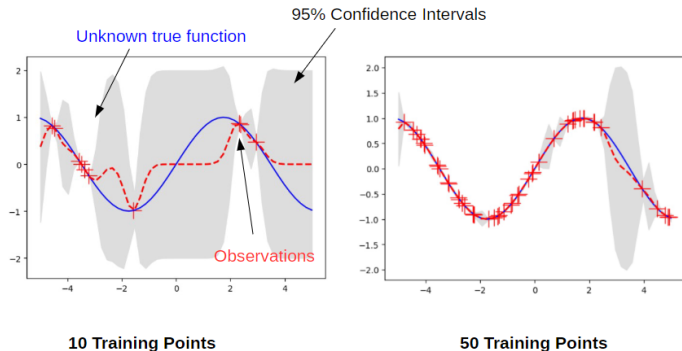
Algorithm 15.1: GP regression

```
1 L = cholesky(K +  $\sigma_y^2$ I);
2  $\alpha = L^T \setminus (L \setminus y)$ ;
3  $\mathbb{E}[f_*] = k_*^T \alpha$ ;
4  $v = L \setminus k_*$ ;
5  $\text{var}[f_*] = \kappa(x_*, x_*) - v^T v$ ;
6  $\log p(y|X) = -\frac{1}{2} y^T \alpha - \sum_i \log L_{ii} - \frac{N}{2} \log(2\pi)$ 
```

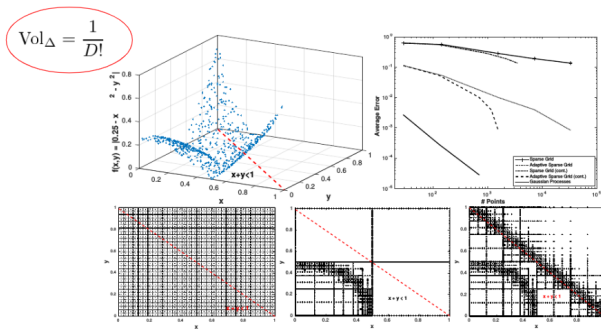
Alg. 15: Numerical more stable.

Prediction & Confidence Intervals

Note: if you don't fix the seed, these pictures vary every time you run the code.



Non-hypercubic domain: GPR versus ASG

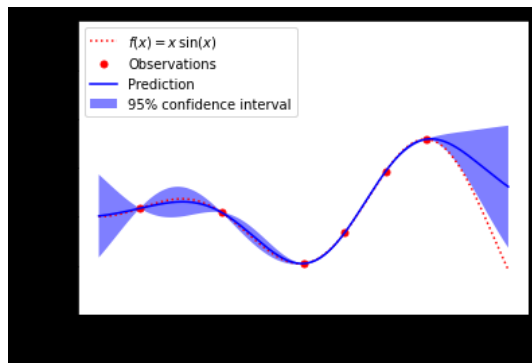


The upper left panel shows the analytical test function evaluated at random test points on the simplex. The upper right panel displays a comparison of the interpolation error for GPs, sparse grids, and adaptive sparse grids of varying resolution and constructed under the assumption that a continuation value exists, (denoted by “cont”), or that there is no continuation value. The lower left panel displays a sparse grid consisting of 32,769 points. The lower middle panel shows an adaptive sparse grid (cont) that consists of 1,563 points, whereas the lower right panel shows an adaptive sparse grid, constructed with 3,524 points and under the assumption that the function outside Δ is not known.

GPR in scikit-learn.org

https://scikit-learn.org/stable/modules/gaussian_process.html

Look at demo/1d_GPR.ipynb



More GP packages (next to scikit-learn.org)

- ▶ GPyTorch

Illustration of noiseless GPR prediction (II)

- ▶ We use a squared exponential kernel, aka Gaussian kernel or RBF kernel.
- ▶ In $1d$, this is given by

$$\kappa(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} (x - x')^2\right)$$

- ▶ Here ℓ controls the horizontal length scale over which the function varies, and σ_f controls the vertical variation.
- ▶ We usually show predictions from the posterior, $p(f_* | X_*, X, f)$.
- ▶ We see (NEXT SLIDE) that the model perfectly interpolates the training data, and that the predictive uncertainty increases as we move further away from the observed data.

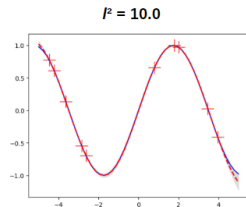
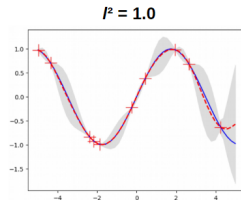
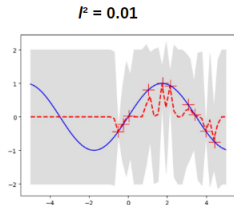
The parameters in the Kernel

cf. `demo/1d_gp_example.ipynb`

$$\kappa(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} (x - x')^2\right)$$

Let $\sigma_f^2 = 1$.

Tuning the parameters by hand is not a good idea in general (in particular in high-dimensional settings).



Gaussian Process Regression (part II)

1. Kernels with noise
2. Hyperparameters / Maximum likelihood
3. Bayesian active learning
4. Option-pricing examples (two separate notebooks)
5. GP classification

Illustration of noiseless GPR prediction (II)

- ▶ We use a squared exponential kernel, aka Gaussian kernel or RBF kernel.
- ▶ In $1d$, this is given by

$$\kappa(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} (x - x')^2\right)$$

- ▶ Here ℓ controls the horizontal length scale over which the function varies, and σ_f controls the vertical variation.
- ▶ We usually show predictions from the posterior, $p(f_* | X_*, X, f)$.
- ▶ We see (NEXT SLIDE) that the model perfectly interpolates the training data, and that the predictive uncertainty increases as we move further away from the observed data.

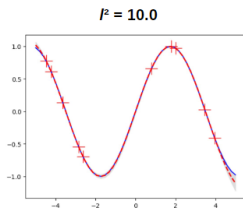
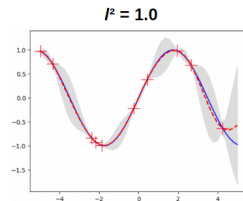
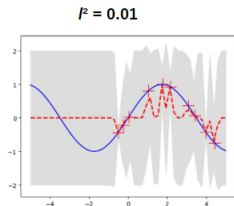
The parameters in the Kernel

cf. `demo/1d_gp_example.ipynb`

$$\kappa(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} (x - x')^2\right)$$

Let $\sigma_f^2 = 1$.

Tuning the parameters by hand is not a good idea in general (in particular in high-dimensional settings).



GPR with noisy data

- ▶ In empirical setups, measurement noise may arise from our inability to control all the influential factors or from irreducible (aleatory) uncertainties.
- ▶ In computer simulations, measurement uncertainty may stem from quasi-random stochasticity, or chaotic behavior.
- ▶ Now let us consider the case where what we observe is a noisy version of the underlying function

$$y = f(x) + \epsilon, \text{ where } \epsilon \sim \mathcal{N}(0, \sigma_y^2)$$

GPR with noisy data (II)

- ▶ In this case (presence of noise), the model is not required to interpolate the data, but it must come “close” to the observed data.
- ▶ The covariance of the observed noisy responses is

$$\text{cov}[y_p, y_q] = \kappa(\mathbf{x}_p, \mathbf{x}_q) + \sigma_y^2 \delta_{pq} \quad \text{where} \quad \delta_{pq} = \mathbb{I}(p = q)$$

- ▶ The second matrix is **diagonal** because we assumed the **noise terms were independently added to each observation**.

GPR with noisy data (III)

- ▶ The joint density of the observed data and the **latent, noise-free function** on the test points is given by

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N} \left(0, \begin{pmatrix} \mathbf{K}_y & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{pmatrix} \right)$$

- ▶ where we are assuming the mean is zero, for notational simplicity.
- ▶ Hence the posterior predictive density is

$$\begin{aligned} p(\mathbf{f}_* \mid \mathbf{X}_*, \mathbf{X}, \mathbf{y}) &= \mathcal{N}(\mathbf{f}_* \mid \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \\ \boldsymbol{\mu}_* &= \mathbf{K}_*^T \mathbf{K}_y^{-1} \mathbf{y} \\ \boldsymbol{\Sigma}_* &= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}_y^{-1} \mathbf{K}_* \end{aligned}$$

Prediction at a single test point

In the case of a single test input, this simplifies as follows

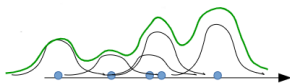
$$p(f_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}) = \mathcal{N}(f_* \mid \mathbf{k}_*^T \mathbf{K}_y^{-1} \mathbf{y}, k_{**} - \mathbf{k}_*^T \mathbf{K}_y^{-1} \mathbf{k}_*)$$

where $\mathbf{k}_* = [\kappa(\mathbf{x}_*, \mathbf{x}_1), \dots, \kappa(\mathbf{x}_*, \mathbf{x}_N)]$

and where $k_{**} = \kappa(\mathbf{x}_*, \mathbf{x}_*) \quad (= 1)$

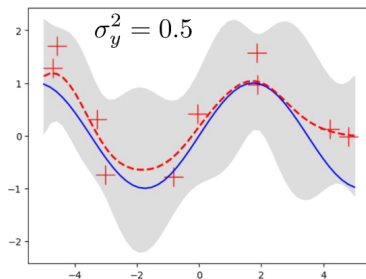
Again, we can write the **posterior mean** as an **expansion of basis functions**:

$$\bar{f}_* = \overbrace{\mathbf{k}_*^T}^{1 \times N} \underbrace{\mathbf{K}_y^{-1}}_{N \times N} \overbrace{\mathbf{y}}^{1 \times N} = \sum_{i=1} \alpha_i \kappa(\mathbf{x}_i, \mathbf{x}_*) \quad \text{where } \alpha = \underbrace{\mathbf{K}_y^{-1} \mathbf{y}}_{\text{from training data}}$$



Some Plots

cf. `demo/1d_gp_example.ipynb`



- ▶ Even in the regions where you have data, there is still uncertainty.
- ▶ In the noise-free version of GPR, the uncertainty is 0 at observation points.
- ▶ But we still have the same properties as before: where we have data, we are more certain compared to the case where we have no data.

Noise improves numerical stability

- ▶ It is common to use **small noise** even if there is not any in the data.
- ▶ Cholesky fails when covariance is close to being semipositive definite.
- ▶ **Adding a small noise improves numerical stability.**
- ▶ It is known as the “jitter” or as the “nugget” in this case.

“Learning” the kernel parameters

- ▶ To estimate the kernel parameters, we could use exhaustive search over a discrete grid of values, with validation loss as an objective, but this can be quite slow.
- ▶ Here we consider an empirical Bayes approach, which will allow us to use continuous optimization methods, which are much faster.
- ▶ In particular, we will maximize the marginal likelihood.

“Learning” the kernel parameters

Marginal likelihood: $p(\mathbf{y} \mid \mathbf{X}) = \int p(\mathbf{y} \mid \mathbf{f}, \mathbf{X}) p(\mathbf{f} \mid \mathbf{X}) d\mathbf{f}$.

Since $p(\mathbf{f} \mid \mathbf{X}) = \mathcal{N}(\mathbf{f} \mid \mathbf{0}, \mathbf{K})$,

and $p(\mathbf{y} \mid \mathbf{f}) = \prod_i \mathcal{N}(y_i \mid f_i, \sigma_y^2)$,

the (log-) marginal likelihood is given by

$$\log p(\mathbf{y} \mid \mathbf{X}) = \log \mathcal{N}(\mathbf{y} \mid \mathbf{0}, \mathbf{K}_y) = -\frac{1}{2} \mathbf{y} \mathbf{K}_y^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_y| - \frac{N}{2} \log(2\pi)$$

where:

- ▶ **1st term**: data fit term
- ▶ **2nd term**: a model complexity term
- ▶ **3rd term**: a constant.

Maximizing the the marginal likelihood

- ▶ Let the kernel parameters (also called **hyper-parameters**) be denoted by θ
- ▶ One can show that the following holds.

$$\begin{aligned}\frac{\partial}{\partial \theta_j} \log p(\mathbf{y} \mid \mathbf{X}) &= \frac{1}{2} \mathbf{y}^T \mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \mathbf{K}_y^{-1} \mathbf{y} - \frac{1}{2} \text{tr} \left(\mathbf{K}_y^{-1} \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right) \\ &= \frac{1}{2} \text{tr} \left((\alpha \alpha^T - \mathbf{K}_y^{-1}) \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right)\end{aligned}$$

where $\alpha = \mathbf{K}_y^{-1} \mathbf{y}$.

Computational complexity:

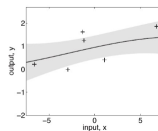
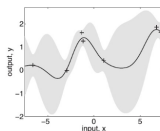
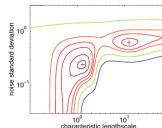
- ▶ It takes $O(N^3)$ time to compute $\mathbf{K}_y^{-1}$
- ▶ $O(N^2)$ time per hyper-parameter to compute the gradient.

Careful: Different optima correspond to different interpretations/beliefs

- We use the SE kernel

$$\kappa_{y}(x_p, x_q) = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} (x_p - x_q)^2\right) + \sigma_y^2 \delta_{pq}$$

- with $\sigma_f^2 = 1$
- We plot $\log p(y | \mathbf{X}, \ell, \sigma_y^2)$ (where $\mathbf{X}$ and y are the 7 data points shown) as we vary ℓ and σ_y^2 .
- The two local optima are indicated by + .
- The bottom left optimum corresponds to a **low-noise, short-length scale solution**.
- The top right optimum corresponds to a **high-noise, long-length scale solution**.
- With only 7 data points, there is not enough evidence to confidently decide which is more reasonable.



An example: noise and optimization

```
import numpy as np
from matplotlib import pyplot as plt
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C

np.random.seed(1)

def f(x):
    """The function to predict."""
    return x * np.cos(x)*np.sin(x)

# Mesh the input space for evaluations of the real function,
# the prediction and its MSE
x = np.atleast_2d(np.linspace(0, 10, 1000)).T

# Instantiate a Gaussian Process model
kernel = RBF()
#kernel = C(1.0, (1e-3, 1e3)) * RBF(10, (1e-2, 1e2))

# now the noisy case
X = np.linspace(0.1, 9.9, 20)
X = np.atleast_2d(X).T

# Observations and noise
y = f(X).ravel()
dy = 0.5 + 1.0 * np.random.random(y.shape)
noise = np.random.normal(0, dy)
y += noise
```

Cont.

```
# Instantiate a Gaussian Process model
gp = GaussianProcessRegressor(kernel=kernel, alpha=dy ** 2,
                              n_restarts_optimizer=10)

# Fit to data using Maximum Likelihood Estimation of the parameters
gp.fit(X, y)

# Make the prediction on the meshed x-axis (ask for MSE as well)
y_pred, sigma = gp.predict(x, return_std=True)

# Plot the function, the prediction and the 95% confidence interval based on the MSE
plt.figure()
plt.plot(x, f(x), 'r:', label=u'$f(x) = x\, \sin(x)$')
plt.errorbar(X.ravel(), y, dy, fmt='r.', markersize=10, label=u'Observations')
plt.plot(x, y_pred, 'b-', label=u'Prediction')
plt.fill(np.concatenate([x, x[::-1]]),
        np.concatenate([y_pred - 1.9600 * sigma,
                        (y_pred + 1.9600 * sigma)[::-1]]),
        alpha=.5, fc='b', ec='None', label='95% confidence interval')
plt.xlabel('$x$')
plt.ylabel('$f(x)$')
plt.ylim(-10, 20)
plt.legend(loc='upper left')
plt.show()
```

A multi-dimensional example

```
import numpy as np
from matplotlib import pyplot as plt
import pickle
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C

np.random.seed(1)
# Test function
def f(x):
    """The 2d function to predict."""
    return np.sin(x[0]) * np.cos(x[1])

# generate training data
n_sample = 100 #points
dim = 2 #dimensions

X = np.random.uniform(-1., 1., (n_sample, dim))
y = np.sin(X[:, 0:1]) * np.cos(X[:, 1:2]) + np.random.randn(n_sample, 1) * 0.005

# Instantiate a Gaussian Process model
kernel = RBF()
#kernel = C(1.0, (1e-3, 1e3)) * RBF(10, (1e-2, 1e2))

gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=9)

# Fit to data using Maximum Likelihood Estimation of the parameters
gp.fit(X, y)
# Make the prediction on the meshed x-axis / training points
y_pred, sigma = gp.predict(X, return_std=True)
```

Cont.

```
#Compute MSE
mse = 0.0
n_sample_test=50
Xtest1 = np.random.uniform(-1., 1., (n_sample_test, dim))
y_pred1, sigma = gp.predict(Xtest1, return_std=True)
for g in range(len(Xtest1)):
    delta = abs(y_pred1[g] - f(Xtest1[g]))
    mse += delta

mse = mse/len(y_pred)
print(".....")
print(" The MSE is ", mse[0])
print(".....")

# Important -- save the model to a file
with open('2d_model.pkl', 'wb') as fd:
    pickle.dump(gp, fd, protocol=pickle.HIGHEST_PROTOCOL)
    print("data written to disk")

# Load the model and do predictions
with open('2d_model.pkl', 'rb') as fd:
    gm = pickle.load(fd)
    print("data loaded from disk")
```

```
# generate training data
n_test = 50
dim = 2
Xtest = np.random.uniform(-1., 1., (n_test, dim))
y_pred_test, sigma_test = gm.predict(Xtest, return_std=True)

MSE2 = 0
for a in range(len(Xtest)):
    delta = abs(y_pred_test[a] - f(Xtest[a]))
    MSE2 +=delta

MSE2 = MSE2/len(Xtest)
print(".....")
print(" The MSE 2 is ", MSE2[0])
print(".....")
```


Stationary versus non-stationary Kernels

Two categories of kernels can be distinguished:

- ▶ **Stationary kernels:**

- ▶ They depend only on the **distance** of two data points and **not on their absolute values** $k(x_i, x_j) = k(d(x_i, x_j))$ and are thus invariant to translations in the input space.
- ▶ Stationary kernels can further be subdivided into **isotropic** and **anisotropic** kernels, where isotropic kernels are also invariant to rotations in the input space.

- ▶ **- Non-stationary kernels:**

- ▶ They depend also on the **specific values of the data points**.

Squared Exponential Kernel

The SE kernel has become the de-facto default kernel for GPS.

$$k_{\text{SE}}(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

This is probably because it has some nice properties:

- ▶ It is **universal**, and **you can integrate it against** most functions that you need to.
- ▶ Every function in its prior has **infinitely many derivatives**.
- ▶ It also has only two parameters:
 - ▶ The **lengthscale** ℓ determines the length of the “wiggles” in your function. In general, **you won't be able to extrapolate more than ℓ units away from your data**.
 - ▶ The output **variance** σ^2 **determines the average distance of your function away from its mean**. Every kernel has this parameter out in front; it's just a scale factor.

Pitfalls for the SE kernel

- ▶ Most people who set up a GP regression (or classification) model end up using the SE kernel.
- ▶ They are a **quick-and-dirty solution** that will probably **work pretty well** for interpolating **smooth functions** when N is a multiple of D , and when there are no “kinks” in your function.
- ▶ If your function happens to have a **discontinuity** or is **discontinuous** in its first few **derivatives** (for example, the `abs()` function), then **either your length-scale will end up being extremely short**, and your posterior mean will become zero almost everywhere, or your posterior mean will have “ringing” effects.
- ▶ Even if there are no hard discontinuities, the length-scale will usually end up being determined by the smallest ‘wiggle’ in your function — so you might end up **failing to extrapolate in smooth regions** if there is even a **small non-smooth region** in your data.
- ▶ **If your data is more than two-dimensional, it may be hard to detect this problem.** One indication is if the length-scale chosen by maximum marginal likelihood never stops becoming smaller as you add more data. This is a classic sign of **model misspecification**.

Periodic Kernel



$$k_{\text{Per}}(x, x') = \sigma^2 \exp \left(-\frac{2 \sin^2(\pi |x - x'| / p)}{\ell^2} \right)$$

- ▶ The periodic kernel allows one to model functions which repeat themselves exactly.
- ▶ Its parameters are easily interpretable:
- ▶ The period p simply determines the distance between repetitions of the function.
- ▶ The **length-scale** ℓ determines the length-scale function in the same way as in the SE kernel.

Matérn kernel

- ▶ The Matérn kernel is a stationary kernel and a generalization of the RBF kernel.
- ▶ It has an additional parameter ν which controls the smoothness of the resulting function. It is parameterized by a length-scale parameter $\ell > 0$, which can either be a scalar (isotropic variant of the kernel) or a vector with the same number of dimensions as the inputs (anisotropic variant of the kernel).
- ▶ The kernel is given by:

$$k(x_i, x_j) = \sigma^2 \frac{1}{\Gamma(\nu)2^{\nu-1}} \left(\gamma \sqrt{2\nu} d(x_i/\ell, x_j/\ell) \right)^\nu K_\nu \left(\gamma \sqrt{2\nu} d(x_i/\ell, x_j/\ell) \right)$$

Matérn kernel (II)

- ▶ As $\nu \rightarrow 0$, the Matérn kernel converges to the RBF kernel.
- ▶ When $\nu = 1/2$, the Matérn kernel becomes identical to the absolute exponential kernel.

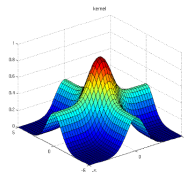
$$k_{\text{mat}}(x, x') = \sigma^2 \left(1 + \sqrt{3} \sum_{i=1}^l \frac{(x_i - x'_i)^2}{\ell_i^2} \right) \exp \left(-\sqrt{3} \sum_{i=1}^l \frac{(x_i - x'_i)^2}{\ell_i^2} \right)$$

- ▶ This are popular choices for learning functions that are not infinitely differentiable (as assumed by the RBF kernel) but at least **once differentiable** ($\nu = 3/2$) (if $\nu = 5/2$, **the kernel is twice differentiable**).

Combining/Adding Kernels

- ▶ Roughly speaking, adding two kernels can be thought of as an **OR** operation.
 - ▶ If you add together two kernels, then the resulting kernel will have high value if either of the two base kernels have a high value.
- ▶ **Linear plus Periodic Kernel**
 - ▶ A linear kernel plus a periodic results in functions which are periodic with increasing mean as we move away from the origin.
- ▶ **Adding across dimensions**
 - ▶ Adding kernels which each depend only on a single input dimension results in a prior over functions which are a sum of one-dimensional functions, one for each dimension. That is, the function $f(x, y)$ is simply a sum of two functions $f_x(x) + f_y(y)$
 - ▶ These kernels have the form:

$$k_{\text{additive}}(x, y, x', y') = k_x(x, x') + k_y(y, y')$$



Automatically choosing Kernels

- ▶ Sometimes, it is not obvious which kernel is appropriate for your problem.
- ▶ In fact, you might decide that choosing the kernel is one of the main difficulties in doing inference
- ▶ Just as you don't know what the true parameters are, you also don't know what the true kernel is.
- ▶ Probably, you should try out a few different kernels at least, and compare their marginal likelihood on your training data.
- ▶ Automatic ways:
 - ▶ <https://arxiv.org/abs/1302.4922> (Structure Discovery in Nonparametric Regression through Compositional Kernel Search)
 - ▶ <https://github.com/jamesrobertlloyd/gp-structure-search>

Summary on GPs

- ▶ For a fairly simple idea, Gaussian processes do tend to work very well on a wide range of topics.
- ▶ The way that the covariance function explicitly encodes the correlations that can be seen in the data means that the user has a lot of control.
- ▶ Even in the simple treatment here we have put quite a lot of effort into making the computations numerically stable and relatively fast.
- ▶ However, there is much more that can be done, including methods for approximation to speed things up significantly.

Pricing options with GPs (BS model)

see

`finance_demo/GP-BS-Pricing_01.ipynb`

- ▶ see `finance_demo/GP-BS-Pricing_02.ipynb`
- ▶ The GP provides analytic derivatives with respect to the input variables

$$\partial_{X_*} \mathbb{E}[\mathbf{f}_* \mid X, Y, X_*] = \partial_{X_*} \boldsymbol{\mu}_{X_*} + \partial_{X_*} K_{X_*, X} \alpha$$

$$\partial_{X_*} K_{X_*, X} = \frac{1}{\ell^2} (X - X_*) K_{X_*, X}$$

$$\alpha = [K_{X, X} + \sigma_n^2 I]^{-1} \mathbf{y}$$

- ▶ Second-order sensitivities $\rightarrow$ diff. wrt. X_*

Pricing options with GPs (Heston)

- ▶ see `finance_demo/GP-Heston-Pricing_03.ipynb`
- ▶ You need to install first Junyan Xu's Python Heston Option Pricing Library
- ▶ `pip install`
`git+https://github.com/junyanxu/Python-HestonOption-Pricer`

Recall: The Heston Model

- ▶ see `finance_demo/GP-Heston-Pricing_03.ipynb`
- ▶ In the following example, the portfolio holds a long position in both a European call and a put option struck on the same underlying, with $K = 100$.
- ▶ We assume that the underlying follows Heston dynamics:

Recall: The Heston Model

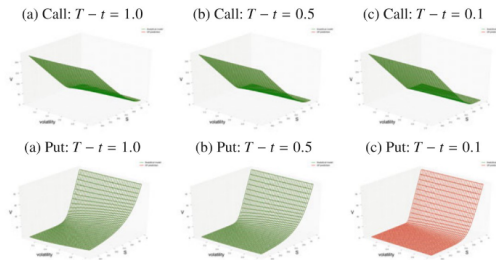
This table shows the values of the parameters for the Heston dynamics and terms of the European Call and Put option contracts.

Parameter description	Symbol	Value
Mean reversion rate	κ	0.1
Mean reversion level	θ	0.15
Vol. of Vol.	σ	0.1
Risk-free rate	r_0	0.002
Strike	K	100
Maturity	T	2.0
Correlation	ρ	-0.9

This table shows the values for the Euler time stepper used for market risk factor simulation.

Parameter description	Symbol	Value
Number of simulation	M	1000
Number of time steps	n_s	100
Initial stock price	S_0	100
Initial variance	V_0	0.1

Some plots



Comparison of the gridded Heston model call (top) and put (bottom) price surfaces at various time to maturities, with the GP estimate.

The GP estimate is observed to be practically identical (slightly below in the first five panels and slightly above in the last one).

Within each column in the figure, the same GP model has been simultaneously fitted to both the Heston model call and put price surfaces over a 30×30 grid of prices and volatilities, fixing the time to maturity. Across each column, corresponding to different time to maturities, a different GP model has been fitted.

The GP is then evaluated out-of-sample over a 40×40 grid, so that many of the test samples are new to the model. This is repeated over various time to maturities.

Gaussian Process Classification

Either go over this notebook, or read the slides first: see
`demo/GPC_classification_step_by_step.ipynb`

Gaussian Process Classification

- ▶ While it is possible to perform multi-class classification with a Gaussian process, we will consider only two classes, labelled as +1 and -1 .
- ▶ The task of the process is then to model the probability that input x belongs to class 1 , which means that the output should be a value between 0 and 1 (inclusive) like all good probabilities.
- ▶ We will arrange this by squashing it using the logistic function

$$P(t^* = 1 \mid a) = \sigma(a) = 1/(1 + \exp(-a))$$

Gaussian Process Classification

- ▶ where $\mathbf{a}$ is the output of the regression GP, and a little care is needed since we are now using $\sigma(\cdot)$ to denote the logistic function, as well as σ_n to denote a hyperparameter and even σ^2 as the variance.
- ▶ Since there are two classes $P(t^* = -1 \mid \mathbf{a}) = 1 - P(t^* = 1 \mid \mathbf{a})$, and so we can write $p(t^* \mid \mathbf{a}) = \sigma(t^* f(\mathbf{x}^*))$.
- ▶ So GP classification consists of finding a GP prior over $f(\mathbf{x})$ (known as the latent function) and then putting this through the logistic function to find a prior on the predicted class, which is:

$$p(t^* = 1 \mid \mathbf{x}, \mathbf{t}, \mathbf{x}^*) = \int \sigma(f(\mathbf{x}^*)) p(f(\mathbf{x}^*) \mid \mathbf{x}, \mathbf{t}, \mathbf{x}^*) df(\mathbf{x}^*).$$

Gaussian Process Classification

- ▶ This is a 1D integral, and so it can be computed numerically.
- ▶ Unfortunately the likelihood function $p(f(x^*) \mid x, t, x^*)$ is not a Gaussian function and so computing that term is intractable.
- ▶ This means that some form of approximation is needed. There are several methods of doing these approximations, including using MCMC.
- ▶ We will consider only the simplest version, which is known as Laplace's approximation.

Laplace Approximation

- ▶ Laplace's approximation is a way to approximate any integral of the form

$$\int \exp(f(\mathbf{x})) d\mathbf{x}$$

which, of course, includes Gaussians.

- ▶ The basic idea is to find the global maximum of the function $f(x)$, which occurs at some x_0 .
- ▶ At this point the gradient of $f(x)$ (which is $\nabla f(x)$) is 0, and so the second-order Taylor expansion around x_0 is (where $\nabla \nabla f(\cdot)$ is the Hessian matrix):

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_0)^T \nabla \nabla f(\mathbf{x}) (\mathbf{x} - \mathbf{x}_0).$$

Laplace Approximation

- Since the logarithm of a Gaussian is a quadratic function, this has a unique maximum, $\rightarrow$ We replace $f(x)$ by $\log f(x)$ and then compute the exponential of this, which tells us that:

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) \exp \left(\frac{1}{2} (\mathbf{x} - \mathbf{x}_0)^T \nabla \nabla \log f(\mathbf{x}) (\mathbf{x} - \mathbf{x}_0) \right).$$

- Normalizing this to make it a Gaussian distribution gives:

$$\begin{aligned} q(f(\mathbf{x}) \mid \mathbf{x}, \mathbf{t}) &\propto \exp \left(-\frac{1}{2} (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^T \mathbf{W} (f(\mathbf{x}) - \hat{f}(\mathbf{x})) \right) \\ &= \mathcal{N} (f(\mathbf{x}) \mid f(\mathbf{x}_0), \mathbf{W}^{-1}), \end{aligned}$$

where $\mathbf{W} = -\nabla \nabla \log f(x)$.

Laplace Approximation

- ▶ In order to compute the Laplace approximation we need to find the value of x_0 and then evaluate the Hessian matrix at that point.
- ▶ Identifying x_0 can be done using the Newton-Raphson iteration, which finds an approximation to solutions of $f(x) = 0$ (in fact, here we want to find $f_0(x) = 0$, but this doesn't change things much) by iterating the computation:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

until the changes are sufficiently small for the required accuracy.

Computing the GPC Posterior

- ▶ Returning to the actual computations that we need for the GP, we had reached the stage of approximating $p(f(x^*) | x, t, x^*)$. Using Bayes' rule we get that:

$$p(f(\mathbf{x}) | \mathbf{x}, \mathbf{t}) = \frac{p(\mathbf{t} | f(\mathbf{x}))p(f(\mathbf{x}) | \mathbf{x})}{p(\mathbf{t} | \mathbf{x})}.$$

- ▶ We are in the lucky situation that the denominator is independent of $f()$, and so can be ignored for the optimization.
- ▶ The first term in the numerator is:

$$p(\mathbf{t} | f(\mathbf{x})) = \prod_{i=1}^N \sigma(f(\mathbf{x}_i))^{t_n} (1 - \sigma(f(\mathbf{x}_i))^{1-t_n}.$$

Computing the GPC Posterior

- We will need to differentiate the log of this expression twice in order to use Equation 1 from before:

$$\begin{aligned}\nabla \log p(\mathbf{t} \mid f(\mathbf{x})) &= \mathbf{t} - \sigma(f(\mathbf{x})) - \mathbf{K}^{-1}f(\mathbf{x}) \\ \nabla \nabla \log p(\mathbf{t} \mid f(\mathbf{x})) &= -\text{diag}(\sigma(f(\mathbf{x}))(1 - \sigma(f(\mathbf{x})))) - \mathbf{K}^{-1}\end{aligned}$$

where $\text{diag}()$ puts the values along the diagonal of a zero matrix, and this term is the $\mathbf{W}$ matrix in Equation 1.

Computing the GPC Posterior

- We now need to find the maximum of $\log p(\mathbf{t} | f(\mathbf{x}))$, for which we construct the Newton-Raphson iteration:

$$\begin{aligned} f(\mathbf{x})^{\text{new}} &= f(\mathbf{x}) - \nabla \nabla \log p(\mathbf{t} | f(\mathbf{x})) \\ &= f(\mathbf{x}) + (\mathbf{K}^{-1} + \mathbf{W})^{-1} (\nabla \log p(\mathbf{t} | f(\mathbf{x})) - \mathbf{K}^{-1} f(\mathbf{x})) \\ &= (\mathbf{K}^{-1} + \mathbf{W})^{-1} (\mathbf{W} f(\mathbf{x}) + \nabla \log p(\mathbf{t} | f(\mathbf{x}))). \end{aligned}$$

- Thus, the Laplace approximation to the posterior probability is:

$$q(f(\mathbf{x}) | \mathbf{x}, \mathbf{t}) = \mathcal{N} \left(\hat{f}, (\mathbf{K}^{-1} + \mathbf{W})^{-1} \right).$$

Computing the GPC Posterior

- Based on this, we can estimate the posterior mean and variance. For the mean, we need to use the fact that at the maximum of $\log p(\mathbf{t} \mid f(\mathbf{x}))$:

$$\hat{f}(\mathbf{x}) = \mathbf{K}(\nabla \log p(\mathbf{t} \mid \hat{f}(\mathbf{x}))),$$

and then the expressions for the mean and variance of the GP regression give us posterior distribution:

$$P(t^* \mid \mathbf{t}, \mathbf{x}, \mathbf{x}^*) \propto \mathcal{N}\left(\mathbf{k}^{*T}(\mathbf{t} - \sigma(f(\mathbf{x}))), k^{**} - \mathbf{k}^{*T}(\mathbf{K} + \mathbf{W}^{-1})^{-1} \mathbf{k}^*\right)$$

Computing the GPC Posterior

- ▶ For the optimization we will also need to calculate the log likelihood and the gradient of it with respect to each hyperparameter, just as we did for GP regression.
- ▶ The log likelihood is:

$$\log p(\mathbf{t} \mid \mathbf{x}, \boldsymbol{\theta}) = \int p(\mathbf{t} \mid f(\mathbf{x})) p(f(\mathbf{x}) \mid \boldsymbol{\theta}) df(\mathbf{x}),$$

and so we again use the Laplace approximation to get:

$$\begin{aligned} \log p(\mathbf{t} \mid \mathbf{x}, \boldsymbol{\theta}) &\approx \log q(\mathbf{t} \mid \mathbf{x}, \boldsymbol{\theta}) \\ &= \log p(\hat{f}(\mathbf{x}) \mid \boldsymbol{\theta}) + \log p\left(\mathbf{t} \mid \hat{f}(\mathbf{x}) - \frac{1}{2} \log \left| \mathbf{W} + \mathbf{K}^{-1} \right| + \frac{N}{2} \log(2\pi)\right) \end{aligned}$$

Computing the GPC Posterior

- ▶ Differentiating this with respect to each of the hyperparameters will lead to two terms, since both $\hat{f}()$ and $\mathbf{K}$ depend on θ .
- ▶ The same matrix identities as for the regression case are useful, and the first part, which is the explicit dependence upon any element of θ is fairly similar to the regression case:

$$\left. \frac{\partial}{\partial \theta_j} \log p(\mathbf{t} \mid \boldsymbol{\theta}) \right|_{\text{explicit}} = \frac{1}{2} \hat{\mathbf{f}}(\mathbf{x})^T \mathbf{K}^{-1} \frac{\partial \mathbf{K}}{\partial \theta_j} \mathbf{K}^{-1} \hat{\mathbf{f}}(\mathbf{x}) - \frac{1}{2} \text{trace} \left((\mathbf{I} + \mathbf{K} \mathbf{W})^{-1} \mathbf{W} \frac{\partial \mathbf{K}}{\partial \theta_j} \right)$$

Computing the GPC Posterior

- We can then use the chain rule to get the other parts:

$$\frac{\partial}{\partial \boldsymbol{\theta}_j} = \frac{\partial}{\partial \hat{f}} \frac{\partial \hat{f}}{\partial \boldsymbol{\theta}_j}, \text{ where:}$$
$$\frac{\partial \hat{f}}{\partial \boldsymbol{\theta}_j} = (\mathbf{I} + \mathbf{W}\mathbf{K})^{-1} \frac{\partial \mathbf{K}}{\partial \boldsymbol{\theta}_j} (\mathbf{t} - \sigma(\hat{f}(\mathbf{x}))),$$

and so we just need to compute:

$$\begin{aligned} & \frac{\partial}{\partial \hat{f}(\mathbf{x}_i)} \log |\mathbf{W} + \mathbf{K}^{-1}| \\ &= ((\mathbf{I} + \mathbf{W}\mathbf{K})^{-1} \mathbf{K})_{ii} \sigma(\hat{f}(\mathbf{x}_i)) (1 - \sigma(\hat{f}(\mathbf{x}_i))) \left(1 - 2\sigma(\hat{f}(\mathbf{x}_i)) \frac{\partial \hat{f}(\mathbf{x}_i)}{\partial \boldsymbol{\theta}_j} \right) \end{aligned}$$

- Note that this includes the third derivative of the $\sigma(\cdot)$ term! Putting these three terms together gives us the whole gradient, ready for the conjugate gradient solver.

Multi-Class GPC

- ▶ It is possible to do multi-class classification with GPs.
- ▶ The basic idea is to use a separate latent function for each class (so that the function $f(x)$ gets c times longer for c classes), looking like:

$$\left(f_1^{C_1}, f_2^{C_1}, \dots, f_n^{C_1}, f_1^{C_2}, f_2^{C_2}, \dots, f_n^{C_2}, \dots, f_1^{C_c}, f_2^{C_c}, \dots, f_n^{C_c}\right).$$

- ▶ The target vector has to be the same dimension, so it will contain a row of $n1$ s where the f for the correct target class are, and 0 everywhere else.
- ▶ The covariance function will then be represented by a set of blocks of the individual covariance matrices.
- ▶ It is also necessary to use soft-max instead of the logistic function to do the 'squashing' of the regression output, which changes the derivatives in the computation of the log likelihood and its gradients.

Example

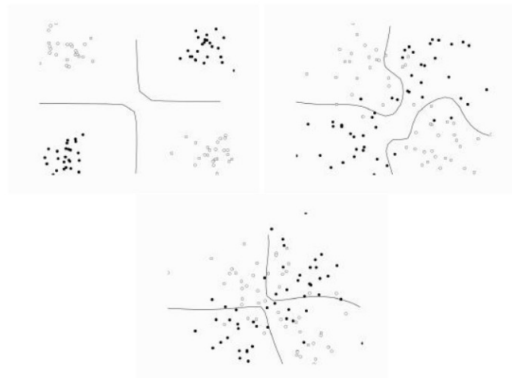


Fig. from Marsland (2014)

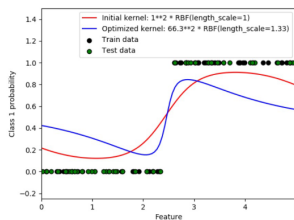
Gaussian process classification on the modified XOR dataset, with standard deviations $\sigma = 0.1$ (left), $\sigma = 0.3$ (middle), $\sigma = 0.4$ (right). The line gives the $p = 0.5$ decision boundary.

GPC in Python — Example 1

https://scikit-learn.org/stable/auto_examples/gaussian_process/plot_gpc.html Run `demo/scikit-gpc.py`

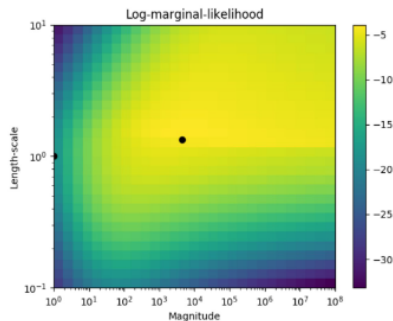
This example illustrates the predicted probability of GPC for an RBF kernel with different choices of the hyper-parameters.

GPC in Python — Example 1



- ▶ This figure shows the predicted probability of GPC with arbitrarily chosen hyper-parameters and with the hyper-parameters corresponding to the maximum log-marginal-likelihood (LML).
- ▶ Note: while the hyper-parameters chosen by optimizing LML have a considerably larger LML, they perform slightly worse according to the log-loss on test data.
- ▶ The figure shows that this is because they exhibit a steep change of the class probabilities at the class boundaries (which is good) but have predicted probabilities close to 0.5 far away from the class boundaries (which is bad) This undesirable effect is caused by the Laplace approximation used internally by GPC.

GPC in Python — Example 1



This figure shows the log-marginal-likelihood for different choices of the kernel's hyperparameters, highlighting the two choices of the hyper-parameters used in the first figure by black dots.

Advanced Topics

<http://proceedings.mlr.press/v37/wilson15.pdf>

- ▶ Ordinary GPs suffer from N^3 scaling.
- ▶ Use a sparse kernel matrix.
- ▶ KISS-GP (Structured Kernel Interpolation).
- ▶ SKI (or KISS-GP) is a great way to scale a GP up to very large datasets (100,000+ data points).
- ▶ <https://people.orie.cornell.edu/andrew/pattern/#SKI>
- ▶ Kernel interpolation for scalable structured Gaussian processes (KISS-GP) was introduced in this paper: <http://proceedings.mlr.press/v37/wilson15.pdf>
- ▶ SKI is asymptotically very fast (nearly linear), very precise (error decays cubically).
- ▶ See, e.g., https://docs.gpytorch.ai/en/v1.1.1/examples/02_Scalable_Exact_GPs/KISSGP_Regression.html

